

Module 9 Structured Note: Checking for Anagrams

1. What this module is about

Module 9 teaches how to check whether two strings are **anagrams**.

An anagram means:

Two strings contain the same letters with the same frequency, but the letters may be arranged in a different order.

Example:

```
decimal  
medical
```

Both strings use the same letters:

```
a c d e i l m
```

So:

```
decimal and medical are anagrams
```

The main idea of this module is:

Order does not matter, but letter count matters.

2. Required ideas from previous modules

Before Module 9, remember these ideas.

C-style strings end with `\0`

A C-style string is a character array ending with the null terminator:

```
'\0'
```

Example:

```
char A[] = "decimal";
```

Memory view:

```
Index: 0 1 2 3 4 5 6 7  
Value: d e c i m a l \0
```

The visible string is:

```
decimal
```

The `\0` is not part of the word. It only tells C where the string ends.

Most string loops use this pattern:

```
for (i = 0; A[i] != '\0'; i++) {  
    // process A[i]  
}
```

This means:

1. Start at index `0`.
2. Check one character at a time.
3. Stop when `\0` is found.

Lowercase letters can be mapped to indexes

For lowercase English letters, we can use this formula:

```
A[i] - 'a'
```

This converts a letter into an array index from `0` to `25`.

Examples:

```
'a' - 'a' = 0  
'b' - 'a' = 1
```

```
'c' - 'a' = 2  
'd' - 'a' = 3  
'e' - 'a' = 4  
'm' - 'a' = 12  
'z' - 'a' = 25
```

This is the same idea used in Module 7 when finding duplicates with a hash table.

3. What is an anagram?

Two strings are anagrams if they contain the same characters the same number of times.

Example 1:

```
A = decimal  
B = medical
```

Both contain:

```
a, c, d, e, i, l, m
```

So they are anagrams.

Example 2:

```
A = listen  
B = silent
```

Both contain:

```
e, i, l, n, s, t
```

So they are anagrams.

Example 3:

```
A = aab  
B = abb
```

These are **not** anagrams.

Why?

```
aab -> a appears 2 times, b appears 1 time  
abb -> a appears 1 time, b appears 2 times
```

They use the same letter types, but not the same letter counts.

4. Order does not matter

In anagrams, the order of letters can change.

Example:

```
decimal  
medical
```

The first string starts with **d**. The second string starts with **m**.

That is okay.

Anagrams do not require the letters to be in the same position.

They only require:

```
same letters + same counts
```

5. Letter frequency matters

Frequency means how many times something appears.

Example:

```
verbose  
observe
```

Both strings contain two **e** letters.

That matters.

If one string has two `e` letters and the other has only one `e`, they are not anagrams.

So when checking anagrams, we cannot only ask:

```
Does this letter exist?
```

We must ask:

```
How many times does this letter exist?
```

6. First rule: lengths must match

Before doing the full anagram check, first check whether both strings have the same length.

Example:

```
A = decimal  
B = medical
```

Both have length `7`, so they may be anagrams.

But:

```
A = decimal  
B = medicalk
```

Lengths:

```
decimal -> 7 letters  
medicalk -> 8 letters
```

They cannot be anagrams because one string has an extra character.

Main rule:

```
If two strings have different lengths, they are not anagrams.
```

7. How to find string length manually

We can find the length by moving through the string until `\0`.

```
int length(char S[]) {  
    int i;  
    for (i = 0; S[i] != '\0'; i++) {  
        // move forward  
    }  
    return i;  
}
```

Example:

```
char S[] = "decimal";
```

Memory:

```
Index: 0 1 2 3 4 5 6 7  
Value: d e c i m a l \0
```

The loop stops at index `7`, so the length is `7`.

Part A: Simple searching method

8. Main idea of the searching method

The simple method checks each character of the first string and searches for it in the second string.

Example:

```
A = decimal  
B = medical
```

We check:

```
Does d exist in medical?  
Does e exist in medical?  
Does c exist in medical?
```

```
Does i exist in medical?  
Does m exist in medical?  
Does a exist in medical?  
Does l exist in medical?
```

If every character from **A** is found in **B**, the strings may be anagrams.

9. Problem with the searching method

The searching method is slow.

Why?

Because for every character in **A**, we may scan many characters in **B**.

That means we use a loop inside another loop.

Basic structure:

```
for (i = 0; A[i] != '\0'; i++) {  
    for (j = 0; B[j] != '\0'; j++) {  
        // search for A[i] inside B  
    }  
}
```

This is called a nested loop.

If the string has **N** characters, the time complexity becomes:

$O(N^2)$

So this method is easy to understand, but it is not the best method for larger strings.

10. Searching method and duplicate letters

The searching method also becomes tricky when duplicate letters are present.

Example:

```
A = aab  
B = abb
```

If we only check whether `a` exists and whether `b` exists, we may wrongly think they are anagrams.

But they are not.

The counts are different:

```
A has two a letters and one b letter.  
B has one a letter and two b letters.
```

So duplicate letters make simple searching harder.

That is why the hash table method is better.

Part B: Hash table / counting array method

11. Main idea of the hash table method

The better method uses a counting array.

For lowercase English letters, we create:

```
int H[26] = {0};
```

This array has 26 slots, one for each lowercase letter.

```
H[0]  -> count of a  
H[1]  -> count of b  
H[2]  -> count of c  
H[3]  -> count of d  
...  
H[25] -> count of z
```

At the start, every value is `0`.

That means:

No letters have been counted yet.

12. Think of H[26] as a letter bank

A simple way to understand this method:

H[26] is like a bank of letters.

First string deposits letters. Second string withdraws letters.

For string A:

```
H[A[i] - 'a']++;
```

This means:

Add one count for this letter.

For string B:

```
H[B[i] - 'a']--;
```

This means:

Remove one count for this letter.

If a count becomes negative, string B used a letter that string A did not have enough of.

So the strings are not anagrams.

13. Why we increment for the first string

Suppose:

```
A = decimal
```

We scan **A** and increase counts.

```
d -> increase count of d
e -> increase count of e
c -> increase count of c
i -> increase count of i
m -> increase count of m
a -> increase count of a
l -> increase count of l
```

In code:

```
for (i = 0; A[i] != '\0'; i++) {
    H[A[i] - 'a']++;
}
```

After this, **H** stores how many times each letter appears in **A**.

14. Why we decrement for the second string

Now suppose:

```
B = medical
```

We scan **B** and decrease counts.

```
m -> decrease count of m
e -> decrease count of e
d -> decrease count of d
i -> decrease count of i
c -> decrease count of c
a -> decrease count of a
l -> decrease count of l
```

In code:

```
for (i = 0; B[i] != '\0'; i++) {
    H[B[i] - 'a']--;
}
```

If `B` contains exactly the same letters as `A`, all counts should go back to `0`.

15. Why negative count means not an anagram

Suppose:

```
A = decimal  
B = medicalk
```

The letter `k` appears in `B`.

But `k` does not appear in `A`.

So when we do:

```
H['k' - 'a']--;
```

The count for `k` goes from `0` to `-1`.

That means:

```
B is trying to use a k that A never provided.
```

So the result is:

```
Not an Anagram
```

16. Main code pattern from Module 9

```
int H[26] = {0};  
  
for (i = 0; A[i] != '\0'; i++) {  
    H[A[i] - 'a'] += 1;  
}  
  
for (i = 0; B[i] != '\0'; i++) {  
    H[B[i] - 'a'] -= 1;  
}
```

```

        if (H[B[i] - 'a'] < 0) {
            printf("Not an Anagram");
            break;
        }
    }

    if (B[i] == '\0') {
        printf("It is an Anagram");
    }
}

```

This is the core logic of Module 9.

17. Full beginner-friendly program

This version includes a length check first.

```

#include <stdio.h>

int length(char S[]) {
    int i;
    for (i = 0; S[i] != '\0'; i++) {
        // move until end of string
    }
    return i;
}

int isAnagram(char A[], char B[]) {
    int H[26] = {0};
    int i;

    // Rule 1: lengths must match
    if (length(A) != length(B)) {
        return 0;
    }

    // Count letters from A
    for (i = 0; A[i] != '\0'; i++) {
        H[A[i] - 'a']++;
    }

    // Remove letters using B
    for (i = 0; B[i] != '\0'; i++) {
        H[B[i] - 'a']--;

        if (H[B[i] - 'a'] < 0) {

```

```

        return 0;
    }
}

return 1;
}

int main() {
    char A[] = "decimal";
    char B[] = "medical";

    if (isAnagram(A, B)) {
        printf("It is an Anagram\n");
    } else {
        printf("Not an Anagram\n");
    }

    return 0;
}

```

Output:

```
It is an Anagram
```

18. Important assumption in this code

This version assumes both strings contain only lowercase letters from `a` to `z`.

It works safely for:

```
decimal
medical
listen
silent
verbose
observe
```

It does not safely handle:

```
Decimal
medical!
```

```
abc123  
hello world
```

Why?

Because this line:

```
H[A[i] - 'a']++;
```

expects `A[i]` to be between `'a'` and `'z'`.

If the character is uppercase, digit, space, or symbol, the index may be invalid.

Part C: Dry runs

19. Dry run 1: `decimal` and `medical`

Strings:

```
A = decimal  
B = medical
```

Start:

```
H[26] = all zeros
```

Step 1: Scan `decimal`

```
d -> H[d] becomes 1  
e -> H[e] becomes 1  
c -> H[c] becomes 1  
i -> H[i] becomes 1  
m -> H[m] becomes 1  
a -> H[a] becomes 1  
l -> H[l] becomes 1
```

Now `H` contains one count for each letter in `decimal`.

Step 2: Scan `medical`

```
m -> H[m] becomes 0
e -> H[e] becomes 0
d -> H[d] becomes 0
i -> H[i] becomes 0
c -> H[c] becomes 0
a -> H[a] becomes 0
l -> H[l] becomes 0
```

No count became negative.

Final result:

It is an Anagram

20. Dry run 2: `decimal` and `medicalk`

Strings:

```
A = decimal
B = medicalk
```

First, check length:

```
decimal -> 7
medicalk -> 8
```

The lengths are different.

So we can immediately say:

Not an Anagram

If we skipped the length check and used the counting method, `k` would still fail because its count would become negative.

When `k` appears:

```
H[k] was 0
H[k] becomes -1
```

That means **B** has an extra **k**.

21. Dry run 3: **verbose** and **observe**

Strings:

```
A = verbose
B = observe
```

These strings both contain repeated letters.

Important counts:

```
verbose -> e appears 2 times
observe -> e appears 2 times
```

Scan **verbose**

```
v -> count becomes 1
e -> count becomes 1
r -> count becomes 1
b -> count becomes 1
o -> count becomes 1
s -> count becomes 1
e -> count becomes 2
```

Scan **observe**

```
o -> count becomes 0
b -> count becomes 0
s -> count becomes 0
e -> count becomes 1
r -> count becomes 0
v -> count becomes 0
e -> count becomes 0
```

No count becomes negative.

Final result:

It is an Anagram

This example shows why counts are important.

22. Dry run 4: aab and abb

Strings:

A = aab
B = abb

Scan aab

a -> H[a] becomes 1
a -> H[a] becomes 2
b -> H[b] becomes 1

Current counts:

a = 2
b = 1

Scan abb

a -> H[a] becomes 1
b -> H[b] becomes 0
b -> H[b] becomes -1

The second b makes the count negative.

That means B has more b letters than A.

Final result:

Not an Anagram

Part D: Why bitwise presence is not enough

23. Review of Module 8 bitwise idea

In Module 8, we used bits to track whether a letter appeared before.

A bit can only be:

0 or 1

That means it can only answer:

Has this letter appeared?

It cannot answer:

How many times has this letter appeared?

24. Why anagrams need counts

Anagrams need exact counts.

Example:

A = aab
B = abb

A bitwise presence check would say:

A has a and b
B has a and b

So it might wrongly think they match.

But the real counts are:

```
A: a = 2, b = 1
B: a = 1, b = 2
```

So they are not anagrams.

Main lesson:

Bitwise presence is good for detecting whether a letter appeared before, but not for checking exact anagram frequency.

For anagrams, use a counting array like:

```
int H[26] = {0};
```

Part E: Safer versions and improvements

25. Safer lowercase-only version

The basic code assumes lowercase letters.

A safer version checks each character before using it as an index.

```
#include <stdio.h>

int length(char S[]) {
    int i;
    for (i = 0; S[i] != '\0'; i++) {
    }
    return i;
}

int isAnagram(char A[], char B[]) {
    int H[26] = {0};
    int i;

    if (length(A) != length(B)) {
        return 0;
    }

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'a' && A[i] <= 'z') {
```

```

        H[A[i] - 'a']++;
    } else {
        return 0;
    }
}

for (i = 0; B[i] != '\0'; i++) {
    if (B[i] >= 'a' && B[i] <= 'z') {
        H[B[i] - 'a']--;

        if (H[B[i] - 'a'] < 0) {
            return 0;
        }
    } else {
        return 0;
    }
}

return 1;
}

```

This version rejects characters that are not lowercase letters.

26. Handling uppercase letters

If you want uppercase and lowercase letters to be treated the same, convert uppercase letters to lowercase first.

Example:

```

Decimal -> decimal
Medical -> medical

```

Conversion pattern:

```

if (A[i] >= 'A' && A[i] <= 'Z') {
    A[i] = A[i] + 32;
}

```

But remember:

Only do this if the string is stored in a mutable character array.

Safe:

```
char A[] = "Decimal";
```

Risky for modification:

```
char *A = "Decimal";
```

27. Using a larger ASCII table

If you want to count all ASCII characters, you can use a larger array:

```
int H[128] = {0};
```

Then you can count using the character value directly:

```
H[A[i]]++;  
H[B[i]]--;
```

This can handle:

```
uppercase letters  
lowercase letters  
digits  
spaces  
symbols
```

But the basic Module 9 method focuses on lowercase `a` to `z` using `H[26]`.

Part F: Complexity analysis

28. Time complexity of searching method

The searching method uses nested loops.

For each character in the first string, it may scan the second string.

So the time complexity is:

$O(N^2)$

This is slower.

29. Time complexity of hash table method

The hash table method scans the first string once and the second string once.

First scan:

$O(N)$

Second scan:

$O(N)$

Total:

$O(N) + O(N) = O(N)$

So the final time complexity is:

$O(N)$

30. Space complexity of hash table method

The hash table uses:

`int H[26]`

This array always has 26 slots.

It does not grow when the string gets longer.

So the extra space complexity is:

$O(1)$

This means constant extra space.

31. Complexity table

Method	Main idea	Time complexity	Extra space
Searching method	Search each character in the other string	$O(N^2)$	$O(1)$
Hash table method	Count letters using <code>H[26]</code>	$O(N)$	$O(1)$

Part G: Common mistakes

32. Mistake 1: Ignoring length

Wrong idea:

I can compare letter counts without checking length first.

The counting method may still catch the error, but the length check is simple and useful.

Correct idea:

If lengths are different, return Not Anagram immediately.

33. Mistake 2: Checking only whether letters exist

Wrong idea:

If both strings contain the same letter types, they are anagrams.

Counterexample:

```
aab  
abb
```

Both contain `a` and `b`, but the counts are different.

Correct idea:

```
Same letters are not enough. Same counts are required.
```

34. Mistake 3: Using bitwise presence for anagrams

Bitwise presence can tell whether a letter appeared.

But it cannot count how many times the letter appeared.

Wrong tool:

```
bitwise presence only
```

Better tool:

```
int H[26] = {0};
```

Anagrams need counts.

35. Mistake 4: Using `H[A[i] - 'a']` without checking input

This is safe only for lowercase letters:

```
H[A[i] - 'a']++;
```

If `A[i]` is uppercase, digit, space, or symbol, the index may be invalid.

Safer version:


```
if (A[i] >= 'a' && A[i] <= 'z') {  
    H[A[i] - 'a']++;  
}
```

36. Mistake 5: Forgetting to stop at `\0`

Wrong:

```
for (i = 0; i < 100; i++) {  
    // check characters  
}
```

Better:

```
for (i = 0; A[i] != '\0'; i++) {  
    // check characters  
}
```

Reason:

`\0` tells us where the actual string ends.

37. Mistake 6: Thinking order matters

Wrong idea:

If the strings are in different order, they cannot be anagrams.

Correct idea:

Order does not matter for anagrams.

Example:

```
decimal  
medical
```

These are anagrams even though the order is different.

Part H: Main code patterns to remember

38. Pattern 1: Find string length

```
int length(char S[]) {  
    int i;  
    for (i = 0; S[i] != '\0'; i++) {  
    }  
    return i;  
}
```

39. Pattern 2: Create a counting array

```
int H[26] = {0};
```

This creates 26 counters for lowercase letters.

40. Pattern 3: Convert a lowercase letter to index

```
A[i] - 'a'
```

Examples:

```
a -> 0  
b -> 1  
d -> 3  
m -> 12  
z -> 25
```

41. Pattern 4: Count letters from first string

```
for (i = 0; A[i] != '\0'; i++) {  
    H[A[i] - 'a']++;  
}
```

Meaning:

Add each letter from A into the count table.

42. Pattern 5: Remove letters using second string

```
for (i = 0; B[i] != '\0'; i++) {  
    H[B[i] - 'a']--;  
  
    if (H[B[i] - 'a'] < 0) {  
        return 0;  
    }  
}
```

Meaning:

Use B to remove letters from the count table.
If any count becomes negative, B has an extra letter.

43. Pattern 6: Full anagram check function

```
int isAnagram(char A[], char B[]) {  
    int H[26] = {0};  
    int i;  
  
    if (length(A) != length(B)) {  
        return 0;  
    }  
  
    for (i = 0; A[i] != '\0'; i++) {  
        H[A[i] - 'a']++;  
    }
```

```

    for (i = 0; B[i] != '\0'; i++) {
        H[B[i] - 'a']--;

        if (H[B[i] - 'a'] < 0) {
            return 0;
        }
    }

    return 1;
}

```

Part I: Full combined example

44. Program to check anagrams

```

#include <stdio.h>

int length(char S[]) {
    int i;
    for (i = 0; S[i] != '\0'; i++) {
    }
    return i;
}

int isAnagram(char A[], char B[]) {
    int H[26] = {0};
    int i;

    if (length(A) != length(B)) {
        return 0;
    }

    for (i = 0; A[i] != '\0'; i++) {
        H[A[i] - 'a']++;
    }

    for (i = 0; B[i] != '\0'; i++) {
        H[B[i] - 'a']--;

        if (H[B[i] - 'a'] < 0) {
            return 0;
        }
    }
}

```

```
        return 1;
    }

    int main() {
        char A[] = "decimal";
        char B[] = "medical";

        if (isAnagram(A, B)) {
            printf("It is an Anagram\n");
        } else {
            printf("Not an Anagram\n");
        }

        return 0;
    }
```

Output:

It is an Anagram

Part J: Practice questions

45. Practice 1

Check whether these are anagrams:

decimal
medical

Expected answer:

It is an Anagram

Reason:

Both contain the same letters with the same counts.

46. Practice 2

Check whether these are anagrams:

verbose
observe

Expected answer:

It is an Anagram

Reason:

Both contain the same letters, including two **e** letters.

47. Practice 3

Check whether these are anagrams:

aab
abb

Expected answer:

Not an Anagram

Reason:

The letter counts are different.

48. Practice 4

Why is bitwise duplicate logic not enough for anagram checking?

Expected answer:

Because bitwise logic only stores whether a letter appeared or not.
Anagrams need exact letter counts.

49. Practice 5

What does this line do?

```
H[A[i] - 'a']++;
```

Expected answer:

It converts the current lowercase letter into an index and increases its count in H.

Example:

If A[i] is 'd', then 'd' - 'a' = 3, so H[3] increases.

50. Final summary

Module 9 teaches how to check whether two strings are anagrams.

The key idea is:

Anagrams have the same letters with the same counts.

The slow method searches each character in the other string.

Time: $O(N^2)$

The better method uses a hash table/counting array:

```
int H[26] = {0};
```

First string adds counts:

```
H[A[i] - 'a']++;
```

Second string removes counts:

```
H[B[i] - 'a']--;
```

If any count becomes negative:

Not an Anagram

If the second string finishes without a negative count:

It is an Anagram

Final complexity:

Time: $O(N)$
Extra space: $O(1)$

The most important lesson:

Anagrams need frequency counting. Presence alone is not enough.